

Implementando Padrões de Teste (Test Patterns)

Disciplina: Teste de Software

Aluna: Roberta Sophia Silva

Matrícula: 839631

Data: 28/05/2026

1. Introdução

Este trabalho parte de um serviço de *checkout* de e-commerce já implementado (o *SUT* — *System Under Test*) para o qual faltava a suíte de testes. O objetivo foi construir testes limpos **desde o início**, aplicando padrões de teste para resolver dois problemas recorrentes: **(a)** a criação de dados de teste de forma legível e flexível (padrões *Object Mother* e *Data Builder*) e **(b)** o isolamento das dependências externas — gateway de pagamento, repositório e envio de e-mail — por meio de *Test Doubles* (*Stubs* e *Mocks*). O código entregue está em `__tests__/builders/UserMother.js`, `__tests__/builders/CarrinhoBuilder.js` e `__tests__/CheckoutService.test.js`.

2. Padrões de Criação de Dados (Builders)

2.1 Por que CarrinhoBuilder e não CarrinhoMother?

O padrão *Object Mother* é ideal para entidades **simples e estáveis**, que variam pouco entre os testes — exatamente o caso do `User`. Por isso o `UserMother` expõe apenas métodos fixos: `umUsuarioPadrao()` e `umUsuarioPremium()`. Já o `Carrinho` é um objeto **complexo e combinatório**: pode ter diferentes usuários, um ou vários itens, valores distintos ou estar vazio. Modelar isso com um *Object Mother* levaria a uma **explosão de métodos** (`umCarrinhoPremiumComDoisItens()`, `umCarrinhoVazioDeUsuarioComum()`, e assim por diante), cada combinação exigindo um novo método.

O *Data Builder* resolve isso oferecendo um **estado padrão válido** e métodos fluentes que customizam **apenas o que importa** em cada cenário. Em vez de N métodos para N combinações, temos poucos métodos componíveis (`.comUser()`, `.comItens()`, `.comItem()`, `.vazio()`) que se encadeiam livremente.

2.2 Antes × Depois

A diferença fica clara comparando o setup manual de um carrinho premium de R\$ 200,00 com o mesmo setup usando o *Data Builder*:

Antes (setup manual)

```
// monta tudo "na mão", expondo
// detalhes irrelevantes ao cenário
const user = new User(
  2, 'Usuário Premium',
  'premium@email.com', 'PREMIUM');

const itens = [
  new Item('Produto A', 150),
  new Item('Produto B', 50),
];
```

Depois (Data Builder)

```
// declara só o que importa:
// é premium e soma R$ 200,00
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Produto A', 150),
    new Item('Produto B', 50),
  ])
  .build();
```

```
const carrinho = new Carrinho(user, item)
// ... e repetir isso em cada teste
```

```
// um carrinho válido padrão fica:
// new CarrinhoBuilder().build()
```

2.3 Como o Builder melhora legibilidade e manutenção

- **Intenção explícita:** o teste mostra apenas o que é relevante para o cenário (usuário premium, total de R\$ 200). O resto fica escondido nos *defaults*, combatendo diretamente o *test smell* de **Setup Obscuro**.
- **API fluente:** o encadeamento (cada método retorna `this`) lê-se quase como uma frase, e a ausência de chamadas (ex.: não chamar `.vazio()`) também comunica intenção.
- **Manutenção centralizada:** se o construtor do `Carrinho` mudar, corrige-se apenas o `build()` do builder — e não dezenas de testes que montavam o objeto manualmente.
- **Menos ruído, menos erro:** dados irrelevantes deixam de poluir cada teste, reduzindo a chance de copiar/colar um setup errado.

3. Padrões de Test Doubles (Mocks vs. Stubs)

Para a análise foi escolhido o teste de **sucesso de um cliente Premium** (Etapa 5), em `__tests__/CheckoutService.test.js`. Nesse cenário, um carrinho premium de R\$ 200,00 deve receber 10% de desconto (cobrança de R\$ 180,00), ser persistido e gerar um e-mail de confirmação.

3.1 Quem é Stub e quem é Mock

Dependência	Tipo de Double	Papel no teste
GatewayPagamento	Stub	Devolve { success: true } só para o fluxo prosseguir.
PedidoRepository	Stub	Devolve um pedido salvo (com id) para alimentar o passo seguinte.
EmailService	Mock	É o alvo da verificação: foi chamado? quantas vezes? com quais argumentos?

```
// Stubs – controlam o fluxo/retorno
const gatewayStub = { cobrar: jest.fn().mockResolvedValue({ success: true }) };
const repositoryStub = { salvar: jest.fn().mockImplementation(
  async (p) => new Pedido(99, p.carrinho, p.totalFinal, p.status) );

// Mock – verificamos a interação
const emailMock = { enviarEmail: jest.fn().mockResolvedValue(undefined) };

// Assert de comportamento:
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, '4111-1111-1111-1111');
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
```

```
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com', 'Seu Pedido foi Aprovado!', 'Pedido 99 no valor de R$180');
```

3.2 Por que o Gateway é (principalmente) Stub e o Email é Mock

A distinção segue a ideia de **Verificação de Estado × Verificação de Comportamento** (Martin Fowler, *Mocks Aren't Stubs*). O `GatewayPagamento` entra como **Stub** porque o que nos interessa dele é o **retorno** que ele injeta no fluxo: dizer “o pagamento deu certo” permite que o restante do `processarPedido` aconteça. Ele é uma **entrada indireta** — controlamos o estado do sistema através dele.

Observação (o “principalmente”): o `cobrar` também é verificado com `toHaveBeenCalled(180, ...)` para confirmar o desconto. Ou seja, ele atua majoritariamente como Stub (controla o fluxo), mas tem um toque de Mock nessa asserção pontual — o que é comum, já que `jest.fn()` registra as chamadas de qualquer forma.

Já o `EmailService` é um **Mock** porque o envio do e-mail é um **efeito colateral** — uma **saída indireta** do sistema que não se reflete no valor de retorno do método. A única maneira de garantir que “o cliente foi notificado, com o destinatário, o assunto e o corpo corretos” é **verificar a interação** com o dublê. Por isso a asserção recai sobre *como* e *com o quê* ele foi chamado, e não sobre um estado retornado.

No teste de **falha de pagamento** (Etapa 4), o mesmo raciocínio aparece de outra forma: o repositório e o e-mail entram como **Dummies** — não devem nem ser chamados —, e a asserção `expect(pedido).toBeNull()` é uma **verificação de estado** pura, controlada pelo Stub do gateway que retorna `{ success: false }`.

3.3 Qualidade dos testes (padrão AAA)

Ambos os testes seguem o padrão **Arrange, Act, Assert**, sinalizado por comentários, com um cenário por bloco `it`. A suíte cobre os três cenários propostos — **falha** de pagamento (retorno `null`), **desconto** de 10% (cobrança de R\$ 180) e **envio de e-mail** (interação verificada). Rodando `npm test`, os dois testes passam.

```
> npm test

> test-pattern@1.0.0 test
> jest

PASS __tests__/CheckoutService.test.js
  CheckoutService
    quando o pagamento falha
      ✓ deve retornar null (pedido não é criado) (5 ms)
    quando um cliente Premium finaliza a compra
      ✓ deve aplicar 10% de desconto e notificar o cliente por e-mail (3 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.835 s
Ran all test suites.

~/Facul/teste/test-pattern main ?1
>
```

Saída do `npm test`: 1 suíte e 2 testes aprovados.

4. Conclusão

O uso deliberado de Padrões de Teste mudou a natureza da suíte: em vez de cada teste carregar um setup verboso e dependências reais, passamos a ter dados de teste **declarativos** (via *Object Mother* e *Data Builder*) e dependências **controladas** (via *Stubs* e *Mocks*). Isso ataca a raiz de *test smells* clássicos — o **Setup Obscuro** some quando o builder esconde o irrelevante, e os **Testes Frágeis** deixam de existir quando isolamos o SUT de serviços externos como gateways e e-mail.

A escolha consciente entre Stub e Mock também torna cada teste mais honesto sobre o que está verificando: estado (o que o método devolve) ou comportamento (com quem o método interagiu). O resultado é uma suíte rápida, isolada, legível e sustentável — testes que de fato protegem a lógica de negócio e que envelhecem bem à medida que o sistema cresce, em vez de oferecer apenas uma falsa sensação de cobertura.